

APP_I2CEEPROM 设计文档

TX32F01 仿真加密 I²C EEPROM 固件

Codex 生成

2026 年 5 月 28 日

目录

1	项目目标	3
1.1	一句话总览	3
1.2	为什么是这个芯片	3
1.3	三层架构	3
2	硬件接线	4
2.1	引脚分配（避开 SWD 引脚）	4
2.2	典型外部接线图	4
3	地址空间与特殊区	4
3.1	Flash 物理布局	4
3.2	Host 可见的逻辑布局	4
3.3	加密区设计细节	5
3.4	计数器实现	5
4	I²C 协议时序	6
4.1	基本写时序	6
4.2	随机读时序（重启动）	6
4.3	连续读时序	6
4.4	Clock Stretching: 覆盖 Flash 写入	7
4.5	Page Write 边界回卷	7
5	协议状态机	7
5.1	状态流程图	8
6	加密区数据流	8
6.1	写入路径	8
6.2	读取路径	8
6.3	攻击模型	9

目录	2
7 容量与性能	9
7.1 固件占用	9
7.2 时间预算	9
8 测试方案	9
8.1 基本探测	9
8.2 读写回环	10
8.3 加密区验证	10
8.4 计数器验证	10
8.5 UART 实时统计	10
9 已知限制	11
10 后续扩展	11
11 文件清单	11

1 项目目标

1.1 一句话总览

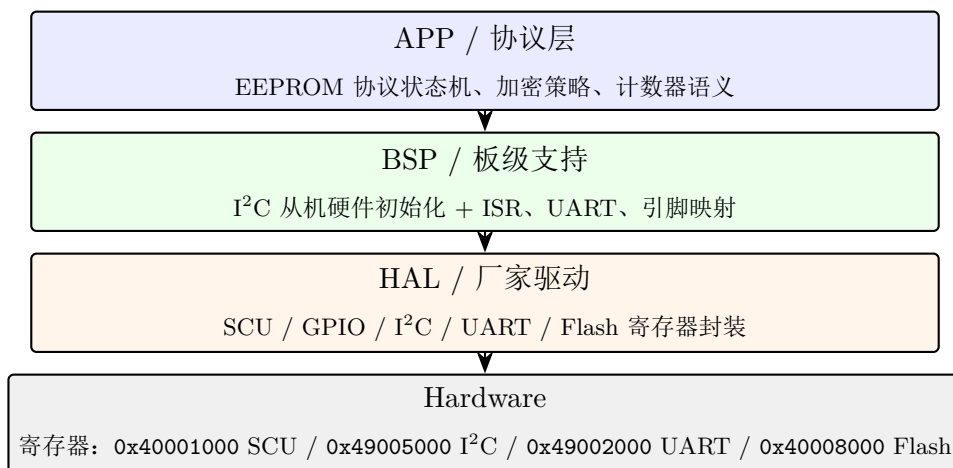
把一颗 TX32F01（Cortex-M0、24 MHz、32 KB Flash、4 KB SRAM）做成一个 **16 KB I²C EEPROM 仿真器**，从总线视角看与 24LC128 兼容，并在内部增加两个真实 EEPROM 不具备的杀手特性：

1. **透明 AES-128-CTR 加密区**：地址 0x3F00–0x3FFB（252 字节）写入 Flash 前自动加密；读出时自动解密给主控。把芯片焊下来用编程器读 **Flash**，看到的是密文。
2. **单调递增计数器**：地址 0x3FFC–0x3FFF（4 字节，big-endian）。读：返回当前计数值；写：忽略数据，计数器 +1。用于固件防回滚、反重放攻击、序号管理。

1.2 为什么是这个芯片

这颗 MCU 的算力和资源（24 MHz、32 KB / 4 KB）与 1980 年代末期高端 EEPROM 控制器接近。把这种”小芯片做储存控制器”的活做出来，比再写一个 PWM 闪灯 demo 工程含金量高得多——而且复制思路就能做出 SPI NOR / 1-Wire EEPROM / 智能卡等一整族”仿真型”产品。

1.3 三层架构



这套分层和 APP_SPINOR 完全一致，是上次项目验证过的范式。

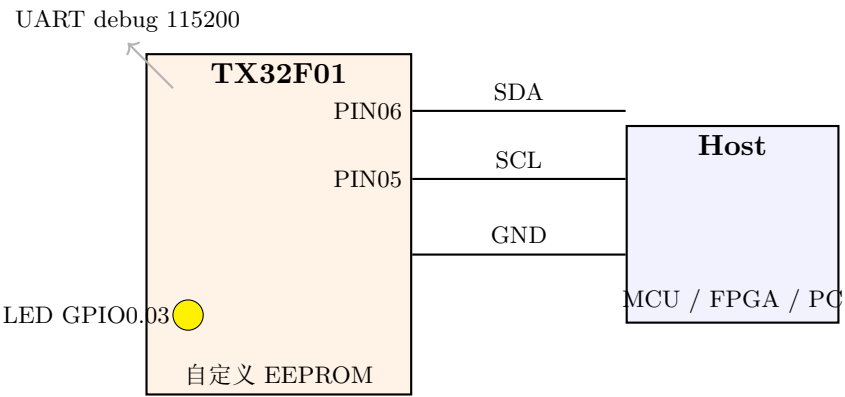
2 硬件接线

2.1 引脚分配（避开 SWD 引脚）

信号	TX32F01 引脚	说明
I ² C SCL	GPIO1.PIN05	同步时钟，主机驱动
I ² C SDA	GPIO1.PIN06	双向数据线
UART TX	GPIO3.PIN07	调试日志，115200 8N1
UART RX	GPIO3.PIN06	未用
LED	GPIO0.PIN03	500 ms 翻转，心跳指示
SWDIO	GPIO0.PIN00	保留，不可挪用
SWCLK	GPIO0.PIN01	保留，不可挪用

I²C 引脚选择依据：vendor 的 9.IIC/5.Slave_IAP demo 就用这两个引脚作为 I²C 从机配置，已经验证过 AF 路径可用。

2.2 典型外部接线图



SCL/SDA 任意一端各需 4.7kΩ 上拉到 3.3V。多数 PC 端 USB-I²C 桥（CH341、Bus Pirate、FT2232）的板载已经做了上拉，不需额外。

3 地址空间与特殊区

3.1 Flash 物理布局

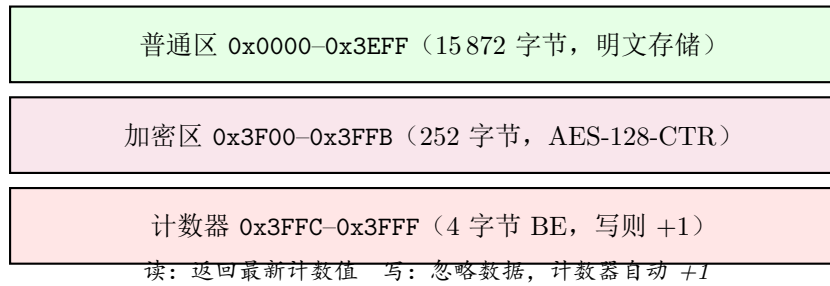
TX32F01 总共 32 KB Flash，划分如下：

物理地址	大小	用途
0x01000000–0x010017FF	6 KB	固件代码 + RO 数据
0x01001800–0x010057FF	16 KB	host 可见 EEPROM
0x01005800–0x01007FFF	10 KB	保留

3.2 Host 可见的逻辑布局

主控看到的 16 KB 地址空间内部分三种区域，但 ** 不需要知道哪是哪 **——读写语义和普通 EEPROM 一致。

读：返回 *Flash* 原值 写：直接编程 *Flash*



3.3 加密区设计细节

密钥派生：固件启动时从 Flash NVR 寄存器读出 0xDIEID0...0xDIEID3 (共 128 bit)，与一个固件版本字面量混合，三轮 XOR + 循环左移生成 16 字节 AES 主密钥。

$$\text{key}[16] = \text{Mix}(\text{DIEID0} \parallel \text{DIEID1} \parallel \text{DIEID2} \parallel \text{DIEID3}; \text{"TX32F01:I2CEE:01"})$$

特性：

- **密钥永不离开芯片：**固件代码不存储密钥，每次启动重新派生。
- **设备唯一：**不同 chip 的 Die ID 不同，密钥不同。A 芯片的 Flash dump 烧到 B 芯片上读出来全是垃圾。
- **固件绑定：**固件版本字面量参与派生，升级固件可强制密钥滚换。

Nonce 设计：AES-CTR 模式，每 16 字节为一个 block。block index 进入 nonce 的低 16 bit，区域 tag "EEPROM01" 占高 64 bit。

$$\text{nonce} = \underbrace{\text{"EEPROM01"}}_{8 \text{ bytes}} \parallel \underbrace{00 \dots 0}_{6 \text{ bytes}} \parallel \underbrace{\text{idx}_{\text{BE}}}_{2 \text{ bytes}}$$

任何字节 A 的加密 / 解密：

1. $\text{idx} = (A - 0x3F00) / 16$ (block 索引)
2. $\text{offset} = (A - 0x3F00) \bmod 16$
3. $\text{stream_byte} = E_K(\text{nonce})[\text{offset}]$
4. $\text{plain} = \text{cipher} \oplus \text{stream_byte}$

XOR 对称，同一函数加密和解密。

3.4 计数器实现

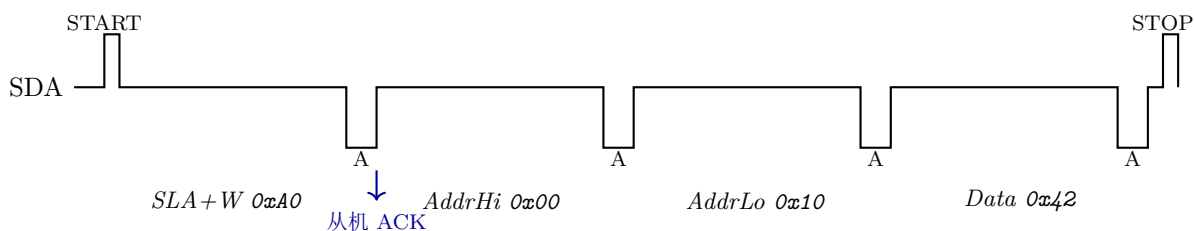
- 4 字节 big-endian 32-bit 无符号整数
- 实际存储位置：Flash 物理地址 0x010057FC–0x010057FF
- **Read** 任意 1 B 落在计数区 → 从存储读出值，按 BE 拆字节

- **Write** 任意值到计数区任意字节 → 计数器 +1，原始 payload 丢弃
- 上电首次读取若为 0xFFFFFFFF（未初始化），按 0 处理

4 I²C 协议时序

4.1 基本写时序

主控写一个字节到地址 0x0010:

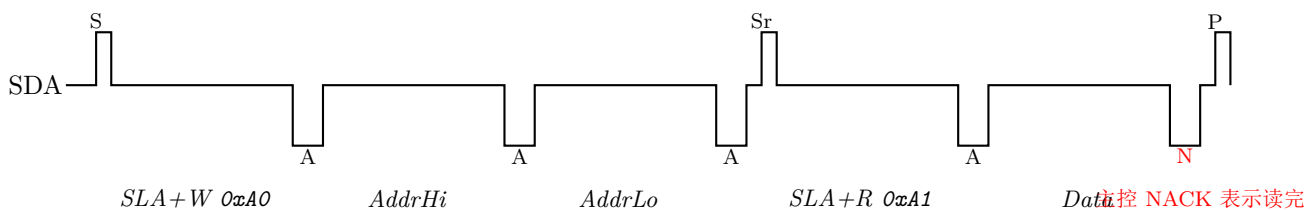


字节解释:

- SLA+W = 0xA0: 7-bit 地址 0x50 左移一位，加 W=0
- 接下来两个字节 = 16-bit word address, big-endian
- Data: 要写入的字节
- STOP 触发实际 Flash 写入（可能耗时 5-40 ms）

4.2 随机读时序（重启动）

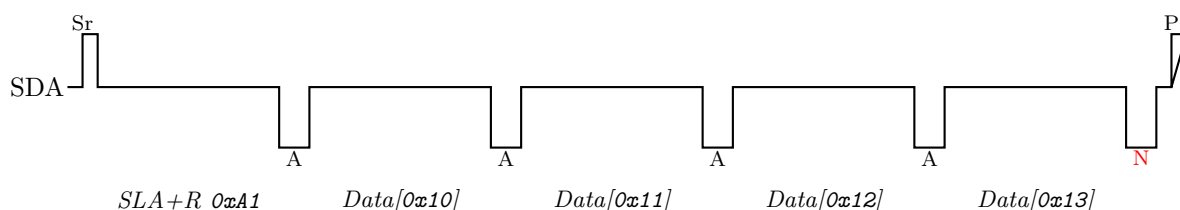
主控读地址 0x0010 处的字节:



关键点：第一阶段是”写地址”（不带数据），重启动 **Sr** 后切到读，从机自动从 0x0010 处取一个字节给主控。主控读完发 NACK 然后 STOP。

4.3 连续读时序

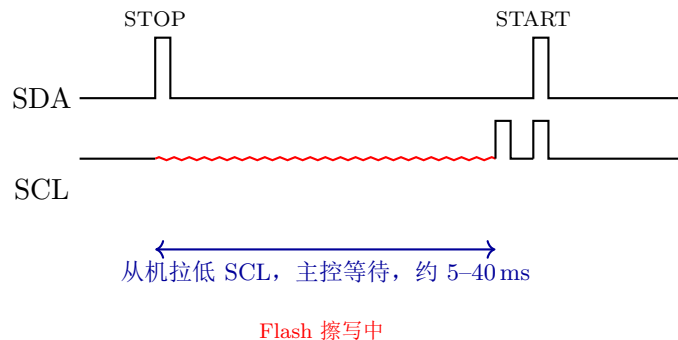
主控随机读后继续 clock，地址自动 +1:



每个字节后主控 ACK 表示还要继续，最后一个字节 NACK + STOP 结束。地址溢出 0x3FFF 后自动从 0x0000 重新开始（wrap-around，与 24LC256 一致）。

4.4 Clock Stretching: 覆盖 Flash 写入

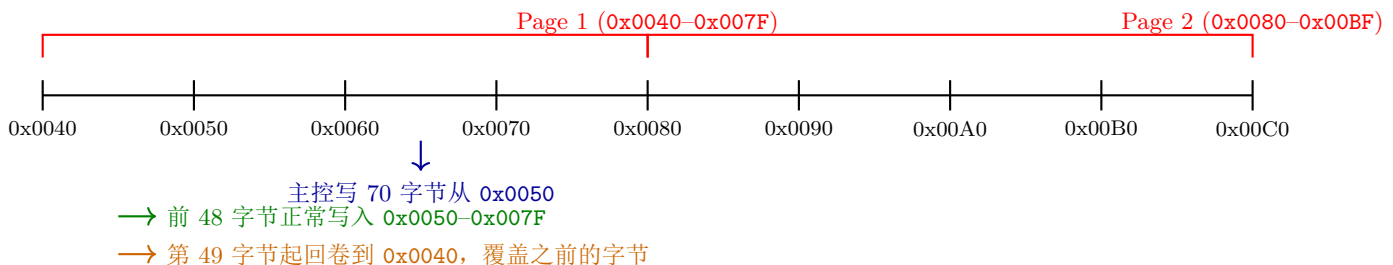
主控在 STOP 后立即想做下一次读，但从机正在写 Flash（5–40 ms）。从机自动拉低 SCL，主控的 START 信号被延迟到从机空闲：



从机怎么做到的：TX32F01 的 I²C 控制器在 CR.SI 位置 1 时自动拉低 SCL，直到 ISR 把 SI 清掉。我们 ISR 在 STOP 事件里如果发现有数据 pending，就先做 Flash 写入再清 SI——主控全程透明等待。

4.5 Page Write 边界回卷

24LC128 的 Page = 64 字节。写超过页边界，地址自动回卷到本页起始：



我们严格遵守这个语义，主控驱动可以利用它实现”环形 page 内更新”等高级模式。

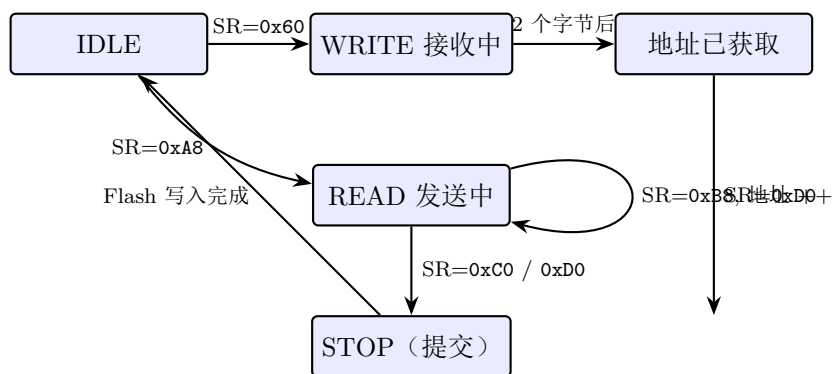
5 协议状态机

每个 I²C 帧分两阶段。从机用 8 位状态码反映总线事件：

SR 值	含义	从机动作
0x60	收到 SLA+W, ACK	重置 page buffer, 准备接收
0x80	收到数据字节, ACK	累积到 page buffer (前 2 字节是地址)
0x88	收到数据字节, NACK	同上, 准备结束
0xA8	收到 SLA+R, ACK	从 addr_ptr 取一字节
0xB8	已发字节, 主控 ACK	继续从 addr_ptr++ 取下一字节
0xC0	已发字节, 主控 NACK	准备 STOP
0xC8	已发最后字节, 主控 NACK	同上
0xD0	收到 STOP	提交 page buffer 到 Flash (若有数据)

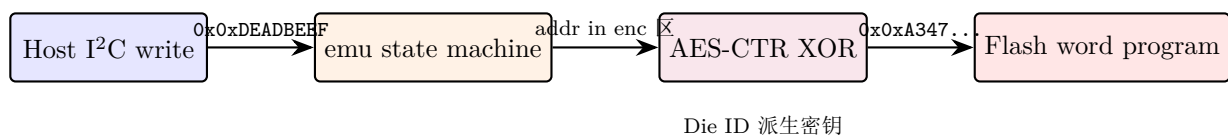
状态机本身极简, 所有”24Cxx 是怎么工作的”业务规则集中在 `on_stop()` 回调里——这正是 BSP / APP 分层的体现。

5.1 状态流程图

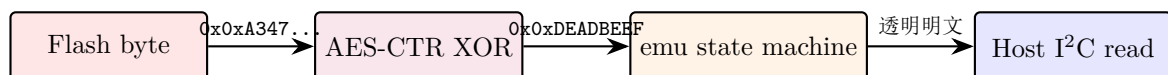


6 加密区数据流

6.1 写入路径



6.2 读取路径



主控读到的永远是明文。任何”芯片焊下来读 Flash”的人看到的永远是密文。

6.3 攻击模型

攻击方式	我们抗得住吗	为什么
拷贝 Flash dump 到另一片 chip	抗	Die ID 不同，密钥不同，解密失败
反复读同一地址寻找规律	抗	CTR nonce 每地址唯一
监听 I ² C 总线	不抗	总线上是明文（本就是设计如此）
SCA / DPA 侧信道攻击	不抗	软件 AES 时间不恒定
完整逆向固件 + Die ID 提取	不抗	拿到密钥派生算法 + Die ID 可复制

定位：抗”无设备的攻击者”，足以防止”把芯片焊下来拷贝 Flash 卖盗版”。不能抗”完全控制设备 + 高级技术”的攻击。

7 容量与性能

7.1 固件占用

模块	ROM 估计
启动代码 + CRT	500 B
AES-128 实现	1.5 KB
存储抽象	0.7 KB
I ² C 状态机	0.5 KB
BSP (I ² C + UART)	1.0 KB
HAL 选用部分	1.5 KB
main + 杂项	0.3 KB
合计	~6 KB

预算 6 KB，留 2 KB 余量。

7.2 时间预算

操作	耗时
AES-128 块加密（单 block）	~145 μ s
1 字节读（明文区）	< 1 μ s
1 字节读（加密区）	~145 μ s
Flash 单 sector 擦除（512 B）	~5 ms
Flash 单 word 编程	~30 μ s
完整 page write（64 B，含擦写）	~8 ms

8 测试方案

8.1 基本探测

Linux 主机 + USB-I²C 桥（CH341A / FT2232H / Bus Pirate）：

```
sudo apt-get install i2c-tools
i2cdetect -y 1
# 应该在地址 0x50 看到设备
```

8.2 读写回环

```
# 写 4 字节到地址 0x0010
i2cset -y 1 0x50 0x00 0x10 0x12 0x34 0x56 0x78 i

# 等 10 ms 让 Flash 完成写入
sleep 0.02

# 用随机读读回
i2cget -y 1 0x50 0x10 b
# 应该返回 0x12 (第一个字节)
```

8.3 加密区验证

1. 主控写 0x0xCAFEBABE 到地址 0x3F00
2. Keil debugger 暂停 CPU，读 Flash 物理地址 0x010057000 (= 0x01001800 + 0x3F00 - wait, 应该是 0x010047000 + 0x3F00 = 0x01004700) 实际: 0x01001800 + 0x3F00 = 0x010057000
3. Flash 中应该是密文，不是 0x0xCAFEBABE
4. 主控读 0x3F00 → 应该读回原始 0x0xCAFEBABE

8.4 计数器验证

```
# 初始读取
i2cget -y 1 0x50 0x3F b ; i2cget -y 1 0x50 0xFC b
# 应该返回 0x00 0x00 (counter == 0)

# 触发 +1
i2cset -y 1 0x50 0x3F 0xFC 0x00 i

# 再读
i2cget -y 1 0x50 0x3F b ; i2cget -y 1 0x50 0xFC b
# counter == 1
```

8.5 UART 实时统计

UART 每秒一行输出:

```
[I2CEE] up=12000ms wr=24 rd=128 bytes=512 addr=0x3F00 counter=7
```

字段: 芯片运行时间、累计写帧数、累计读帧数、累计字节数、最后访问地址、当前计数值。

9 已知限制

- **I²C 最高时钟 400 kHz**: 超过这个频率 ISR 跟不上字节速率。标准 24LC256 fast-mode 也是这个值，所以兼容主流主控。
- **每次 STOP 都触发 Flash 擦写**: 写入吞吐受 Flash 擦写时间限制。多次小写比一次大写慢得多。这是真实 EEPROM 也有的限制。
- **无写保护**: BP 位、WC 引脚等都没实现。后续可以加。
- **加密强度有限**: 密钥派生算法不是 KDF 强度，靠的是“Die ID 保密”。

10 后续扩展

每个都是 200–400 行的独立工作:

1. **多 bank**: 响应多个 I²C 地址，每地址独立存储区。
2. **写保护硬件引脚 (WP)**: GPIO 输入低 = 禁止写入。
3. **访问日志**: 每次访问记录到隐藏区，主控不可见，供取证。
4. **ECDH 协商会话密钥**: 主控和从机用 ECDH 握手，会话内使用一次性密钥。
5. **Flash wear leveling**: 扩到外部 SPI NOR backing, 10 万擦写寿命变 1000 万。

11 文件清单

文件	作用
USER/main.c	入口、初始化、1 秒心跳日志
EEPROM/eeprom_protocol.h	24Cxx 协议常量、区域定义
EEPROM/eeprom_emu.h/.c	协议状态机
EEPROM/eeprom_storage.h/.c	存储抽象 (加密 + counter)
EEPROM/eeprom_crypto.h/.c	AES-128 + CTR 软件实现
BSP/bsp_i2c_slave.h/.c	I ² C 从机硬件 + ISR
BSP/bsp_uart.h/.c	调试 UART
TX32F01_I2CEEPROM.sct	链接脚本
TX32F01_I2CEEPROM.uvprojx	Keil 工程
doc/app_i2ceeprom_design.tex	本文档